`views::to_input`

Document #:	P3137R1
Date:	2024-05-21
Project:	Programming Language C++
Audience:	SG9

Reply-to:	Tim Song <t.canens.cpp@gmail.com>

§ 1 Abstract

This paper proposes the `views::to_input` adaptor that downgrades a source range to an input-only, non-common range.

§ 2 Revision history

— R1: Added `operator-` for `sized_sentinel_for` cases per SG9 feedback. Added additional discussion on the feasibility of splitting this adaptor.

§ 3 Motivation

The motivation for this view is given in [P2760R1] and quoted below for convenience (with the name changed from `as_input` to `to_input`; see the naming discussion below):

> Why would anybody want such a thing? Performance.
>
> Range adaptors typically provide the maximum possible iterator category - in order to maximize functionality. But sometimes it takes work to do so. A few examples:
>
> — `views::join(r)` is common when `r` is, which means it provides two iterators. The iterator comparison for `join` does two iterator comparisons, for both the outer and the inner iterator, which is definitely necessary when comparing two iterators. But if all you want to do is compare `it == end`, you could've gotten away with one iterator comparison. As such, iterating over a common `join_view` is more expensive than an uncommon one.
>
> — `views::chunk(r, n)` has a different algorithm for input vs forward. For forward+, you get a range of `views::take(n)` - if you iterate through every element, then advancing from one chunk to the next chunk requires iterating through all the elements of that chunk again. For input, you can only advance element at a time.
>
> The added cost that `views::chunk` adds when consuming all elements for forward+ can be necessary if you need the forward iterator guarantees. But if you don't need it, like if you're

just going to consume all the elements in order one time. Or, worse, the next adaptor in the chain reduces you down to input anyway, this is unnecessary.

In this way, `r | views::chunk(n) | views::join` can be particularly bad, since you're paying additional cost for `chunk` that you can't use anyway, since `views::join` here would always be an input range. `r | views::to_input | views::chunk(n) | views::join` would alleviate this problem. It would be a particularly nice way to alleviate this problem if users didn't have to write the `views::to_input` part!

This situation was originally noted in [range-v3#704].

During the SG9 discussion, it was also pointed out that adaptors on input-only views never need to cache `begin()` as that can be called only once, which can reduce the space overhead of view pipelines in certain constrained environments when that is not needed.

§ 4 Design

§ 4.1 One adaptor, not two

During the 2024-03-19 SG9 review, the room voted 1/5/7/0/0 in favor of suggestion to split this adaptor into two pieces: one for demoting-to-input and one for make-uncommon, principally on the basis that uses of the two can have distinct rationales and so can benefit from separate annotations.

This direction does not appear viable, however, because the demote-to-input adaptor should have a move-only iterator to discourage misuse. That means that it *cannot* be common because sentinels are required to be semiregular. Losing the opportuntity to diagnose misuses at compile-time does not appear to this author to be worth the marginal benefit in encouraging annotation.

While a separate make-uncommon adaptor is possible, this paper doesn't propose it; the performance benefit from that change alone may not justify a separate adaptor in in the standard.

§ 4.2 Naming

P2760R1 proposes `as_input`, but the two `as_`*`meow`* views we have (`as_const` and `as_rvalue`) are element-wise operations rather than operations on the view itself. So this paper proposes the name `to_input`.

§ 4.3 Properties

`to_input` is conditionally borrowed, input-only (that's its whole point), non-common, and conditionally const-iterable. As usual, wrapping is avoided if possible. The iterator only provides the minimal interface needed for an input iterator.

§ 4.4 Additional operations?

In theory, we could provide all the operations supported by the underlying iterator, and limit our change to the iterator concept. Such operations would be unlikely to see much use: `to_input` is a facility for influencing generic algorithms, and generic algorithms generally have to rely on the concept to determine if an operation can be used. So we do not try to do so here.

§ 5 Wording

This wording is relative to [N4971].

§ 5.1 Addition to <ranges>

Add the following to 26.2 [ranges.syn], header <ranges> synopsis:

```
// [...]
namespace std::ranges {
  // [...]

  // [range.to.input], to input view
  template<input_range V>
    requires view<V>
  class to_input_view;

  template<class V>
  inline constexpr bool enable_borrowed_range<to_input_view<V>> =
      enable_borrowed_range<V>;

  namespace views {
    inline constexpr unspecified to_input = unspecified;
  }

}
```

§ 5.2 to_input

Add the following subclause to 26.7 [range.adaptors]:

§ 26.7.? To input view [range.to.input]

§ 26.7.?.1 Overview [range.to.input.overview]

1 `to_input_view` presents a view of an underlying sequence as an input-only non-common range. [*Note 1:* This is useful to avoid overhead that may be necessary to provide support for the operations needed for greater iterator strength. — *end note*]

2 The name `views::to_input` denotes a range adaptor object (26.7.2 [range.adaptor.object]). Let `E` be an expression and let `T` be `decltype((E))`. The expression `views::to_input(E)` is expression-equivalent to:

.1) — `views::all(E)` if `T` models `input_range`, does not model `common_range`, and does not model `forward_range`;

.2) — Otherwise, `to_input_view(E)`.

§ 26.7.?.2 Class template `to_input_view` [range.to.input.view]

```
template<input_range V>
  requires view<V>
class to_input_view : public view_interface<to_input_view<V>>{
  V base_ = V();                            // exposition only

  template<bool Const>
  class iterator;                           // exposition only

public:
  to_input_view() requires default_initializable<V> = default;
  constexpr explicit to_input_view(V base);

  constexpr V base() const & requires copy_constructible<V> { return base_; }
  constexpr V base() && { return std::move(base_); }

  constexpr auto begin() requires (!simple-view<V>);
  constexpr auto begin() const requires range<const V>;

  constexpr auto end() requires (!simple-view<V>);
  constexpr auto end() const requires range<const V>;

  constexpr auto size() requires sized_range<V>;
  constexpr auto size() const requires sized_range<const V>;
};

template<class R>
  to_input_view(R&&) -> to_input_view<views::all_t<R>>;
```

```
constexpr explicit to_input_view(V base);
```

1 *Effects*: Initializes *base_* with `std::move(base)`.

```
constexpr auto begin() requires (!simple-view<V>);
```

2 *Effects*: Equivalent to:

```
return iterator<false>(ranges::begin(base_));
```

```
constexpr auto begin() const requires range<const V>;
```

3 *Effects*: Equivalent to:

```
return iterator<true>(ranges::begin(base_));
```

```
constexpr auto end() requires (!simple-view<V>);
constexpr auto end() const requires range<const V>;
```

4 *Effects*: Equivalent to:

```
      return ranges::end(base_);
```

```
constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
```

5 *Effects*: Equivalent to:

```
      return ranges::size(base_);
```

§ 26.7.?.3 Class template `to_input_view::iterator` [range.to.input.iterator]

```
namespace std::ranges {
  template<input_range V>
    requires view<V>
  template<bool Const>
  class to_input_view<V>::iterator {
    using Base = maybe-const<Const, V>;                          // exposition only

    iterator_t<Base> current_ = iterator_t<Base>();              // exposition only

    constexpr explicit iterator(iterator_t<Base> current);       // exposition only

  public:
    using difference_type = range_difference_t<Base>;
    using value_type = range_value_t<Base>;
    using iterator_concept = input_iterator_tag;

    iterator() requires default_initializable<iterator_t<Base>> = default;

    iterator(iterator&&) = default;
    iterator& operator=(iterator&&) = default;

    constexpr iterator(iterator<!Const> i)
      requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;

    constexpr iterator_t<Base> base() &&;
    constexpr const iterator_t<Base>& base() const & noexcept;

    constexpr decltype(auto) operator*() const { return *current_; }

    constexpr iterator& operator++();
    constexpr void operator++(int);

    friend constexpr bool operator==(const iterator& x, const sentinel_t<Base>& y);

    friend constexpr difference_type operator-(const sentinel_t<Base>& y, const
        iterator& x)
      requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
    friend constexpr difference_type operator-(const iterator& x, const
        sentinel_t<Base>& y)
      requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

    friend constexpr range_rvalue_reference_t<Base> iter_move(const iterator& i)
      noexcept(noexcept(ranges::iter_move(i.current_)));
```

```
    friend constexpr void iter_swap(const iterator& x, const iterator& y)
      noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
      requires indirectly_swappable<iterator_t<Base>>;
  };
}
```

```
constexpr explicit iterator(iterator_t<Base> current);
```

1 *Effects*: Initializes *current_* with `std::move(current)`.

```
constexpr iterator(iterator<!Const> i)
  requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;
```

2 *Effects*: Initializes *current_* with `std::move(i.current_)`.

```
constexpr iterator_t<Base> base() &&;
```

3 *Returns*: `std::move(current_)`.

```
constexpr const iterator_t<Base>& base() const & noexcept;
```

4 *Returns*: *current_*.

```
constexpr iterator& operator++();
```

5 *Effects:* Equivalent to:

```
        ++current_;
        return *this;
```

```
constexpr void operator++(int);
```

6 *Effects*: Equivalent to: `++*this;`

```
friend constexpr bool operator==(const iterator& x, const sentinel_t<Base>& y);
```

7 *Returns:* `x.current_ == y`.

```
friend constexpr difference_type operator-(const sentinel_t<Base>& y, const iterator&
        x)
  requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
```

8 *Returns:* `y - x.current_`.

```
friend constexpr difference_type operator-(const iterator& x, const sentinel_t<Base>&
        y)
      requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
```

9 *Returns:* `x.current_ - y`.

```
friend constexpr range_rvalue_reference_t<Base> iter_move(const iterator& i)
  noexcept(noexcept(ranges::iter_move(i.current_)));
```

10 *Effects*: Equivalent to: `return ranges::iter_move(i.current_);`

```
friend constexpr void iter_swap(const iterator& x, const iterator& y)
  noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
  requires indirectly_swappable<iterator_t<Base>>;
```

11 *Effects*: Equivalent to: `ranges::iter_swap(x.current_, y.current_);`

§ 6 References

[N4971] Thomas Köppe. 2023-12-18. Working Draft, Programming Languages — C++.
https://wg21.link/n4971

[P2760R1] Barry Revzin. 2023-12-15. A Plan for C++26 Ranges.
https://wg21.link/p2760r1

[range-v3#704] Eric Niebler. 2017. Demand-driven view strength weakening.
https://github.com/ericniebler/range-v3/issues/704